7章
すべてをまとめる

少しばかり間を置きましょう。前の3つの章は長くて濃い内容でした。アプリケーションを接続するためのデータ流通と統合パターンについて学びました。深く入り込み、さまざまなアーキテクチャを管理するためのさまざまなパターンと検討すべき事項について議論しました。また、データと統合の両方を管理するための、より包括的なアプローチについても学びました。

ここからは、すべてをまとめて、アーキテクチャ全体の管理について説明します。まず、クロスドメインにおける相互運用性、データ流通、アプリケーション統合について簡単に復習します。その後、統合パターンを選択する際の評価基準、どのような組み合わせが可能か、ある特定の状況で最も効果的なのは何かなど、さまざまなテーマについて説明します。ドキュメント化、設計、ガバナンス、メタデータ管理、セキュリティなどのベストプラクティスには重なる部分が多いことがおわかりいただけるでしょう。次に、少し趣向を変えて、どのようにインスピレーションを与え、やる気を起こさせ、変化を導くかを探ります。まず、クロスドメインの境界を正しく管理する方法から始め、具体的なソリューション設計について議論します。そして、発見や意味論的一貫性を追求する方法や、このようなアーキテクチャの移行においてチームを適切に導く方法について見ていきます。最後に、組織の変革について考察します。ドメインチームとプラットフォームチームはどのように相互協力すべきなのか、そしてそこからどのようなことを学ぶことができるのでしょうか。この章が終わるころには、大規模なデータ流通と統合のランドスケープを管理することの意味をしっかりと理解しているはずです。データ管理の領域を増やしてランドスケープを拡大する際には、このような知識が重要になります。

この章では、変化に対応するアーキテクチャを構築する際に、エンタープライズアーキテクチャが果たすべき役割について、個人的な知見を交えて説明します。筆者はリードアーキテクトとして、技術設計基準や、ガイドライン、将来アーキテクチャを定義する責任を担っていました。また、エンタープライズア

ーキテクチャのガバナンスプロセスを管理するアーキテクチャレビューボード（ARB）の責任者も務めました。この経験は、こうした組織を活用することで、ステークホルダーとつながったり、やるべき仕事への意欲を高めたりできることを教えてくれました。さらに、組織内のあらゆる勢力や周囲の状況を概観することができました。これからもさまざまな事例をご紹介していきますので、ご期待ください。

7.1　クロスドメイン相互運用性

以前は、データの分類と統合は簡単でした。運用側と分析側を明確に区別していたからです。運用側は、トランザクション処理、プロセスの統合、機能性（ビジネスロジック）の共有に重点を置きます。分析側は、さまざまな（分析）アプリケーション間で同じアプリケーションデータを共有することに重点を置いています。それぞれの側には独自のアーキテクチャがあり、その結果、パターン、ユースケース、ガイダンス、ペルソナ、テクノロジーに差異が生じます。アプリケーションを保守する人々は、一般的に、データプロダクトやAPIを構築する人々と同じであるため、この差異が問題になります。図7-1を見てください。データプロダクトの基盤となるアーキテクチャは、APIがリソースにアクセスするのと同じアーキテクチャです。

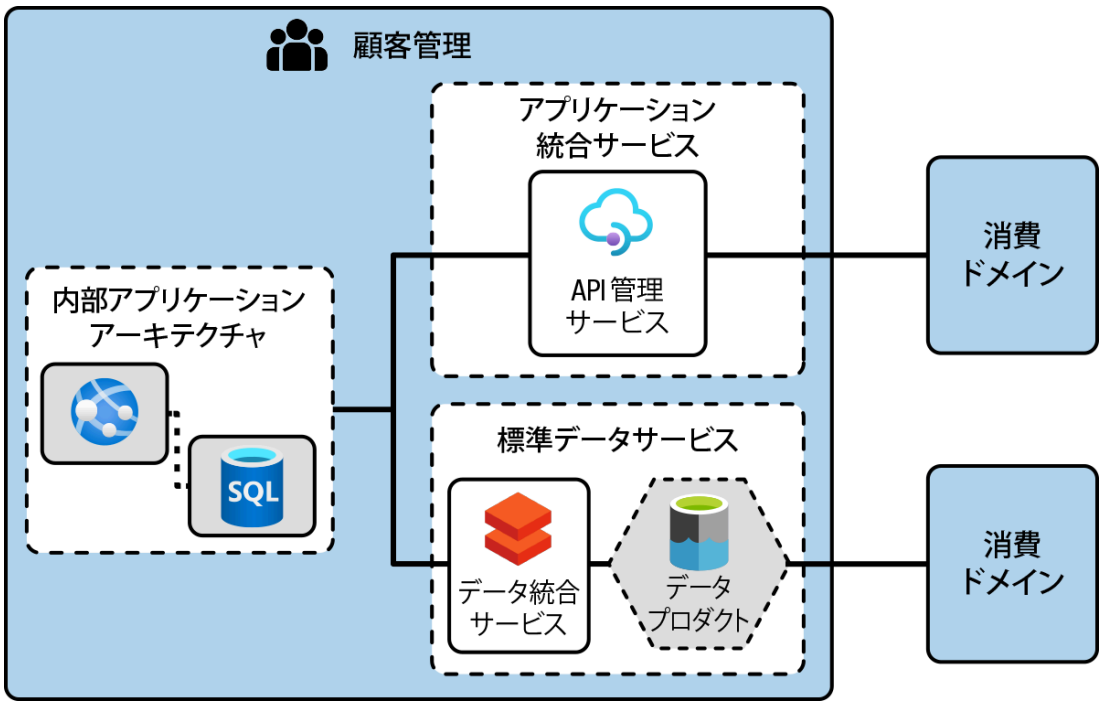

図7-1　データプロダクトとAPIの内部アプリケーションアーキテクチャは基本的に同じ

データ管理と同じことが、アプリケーション開発の入力となる、コンテキストやユビキタス言語にもあてはまります。データプロダクト、API、イベントは、この同じドメインコンテキストからユビキタス言語を継承します。では、このような重なりを考慮しても、分析プレーンと運用プレーンを切り離して管理することに意味があるのでしょうか。そこで、両者を管理するための包括的なアプローチを提案します。このアプローチは、データの管理と統合の関心事を整合させることから始まります。データプロダクト、API、イベントを一緒に管理するという考え方をさらに深く掘り下げる前に、これまでに学んだことを簡単に振り返ってみましょう。

7.1.1　簡単な振り返り

2章では、ドメインがほかのドメインとどのように通信するかについて概要を説明しました。内部ドメインとアプリケーションの詳細は、ほかのドメインから隠蔽されなければならず、ドメインはほかのドメインのため標準インタフェースを公開して、作業が簡単になるようにしなければならないことを学びました。3章では、ドメイントポロジーについて学び、ランディングゾーンとブループリントでドメインをどのようにサポートできるかを知りました。

4章では、今日のデータ駆動型の世界と、読み出しに最適化されたデータへの需要の高まりについて学びました。そこでデータプロダクトの出番です。データプロダクトは、同じデータを何度も効率的に読み出すために使用できます。また、CQRSや、同期的に更新される読み出し専用のデータコピーの作成についても学びました。これは、トランザクションアプリケーションで見られるような正確なデータを提供するわけではありませんが、データコンシューマが大量のデータを読み出すことを可能にしてくれます。

次に、5章ではAPI管理について学びました。APIはリアルタイムの同期型通信のためのものです。レガシーシステムと最新型のアプリケーション間の通信に使用できます。このパターンは、トランザクションクエリや強一貫性の（正確な）データのために使用されます。

6章では、イベント駆動型アーキテクチャを取り上げました。イベント駆動型アーキテクチャでは、リアルタイムにデータを変換し、その結果をほかのアプリケーションに通知することができます。イベントストリーミングは対話ではなく、リスナーにデータを送信する一方通行の通信チャネルです。「アプリケーションからどのようなデータを要求したり取り出したりできるか」というの

ではなく、「通り過ぎていくどのデータが自分にとって有用か」というモデルです。ストリーミングを使えば、データに素早く反応することができます。

データプロダクト、API、イベントの基盤となるアーキテクチャはそれぞれ異なります。異なる目的のために構築されていますが、これらは重なり合っており、組み合わせることもできます。柔軟性を考えれば素晴らしいことなのですが、一方で、トレードオフや考慮すべき点が多いため、エンジニアやアーキテクトにとっては、設計が困難なものになっています。そこで、次の項では設計例を提案します。この設計には、ソリューションを効果的に開発するためのベストプラクティスが含まれています。その後、筆者が大企業のリードアーキテクトとして働く中で学んだことを紹介します。

7.1.2　データ流通とアプリケーション統合の比較

データプラットフォームと統合サービスをドメインに整合させる場合、さまざまなパターンに応じたアーキテクチャガイダンスを整備することをお勧めします。そのために、図7-2に示すようなブループリントをデータプロジェクトと統合プロジェクトの両方に対して作成することを提案します。この図には、前の章で説明した主要なパターンがすべて含まれています。さまざまなパターンが互いに補完し合っています。この点については後で詳しく説明します。

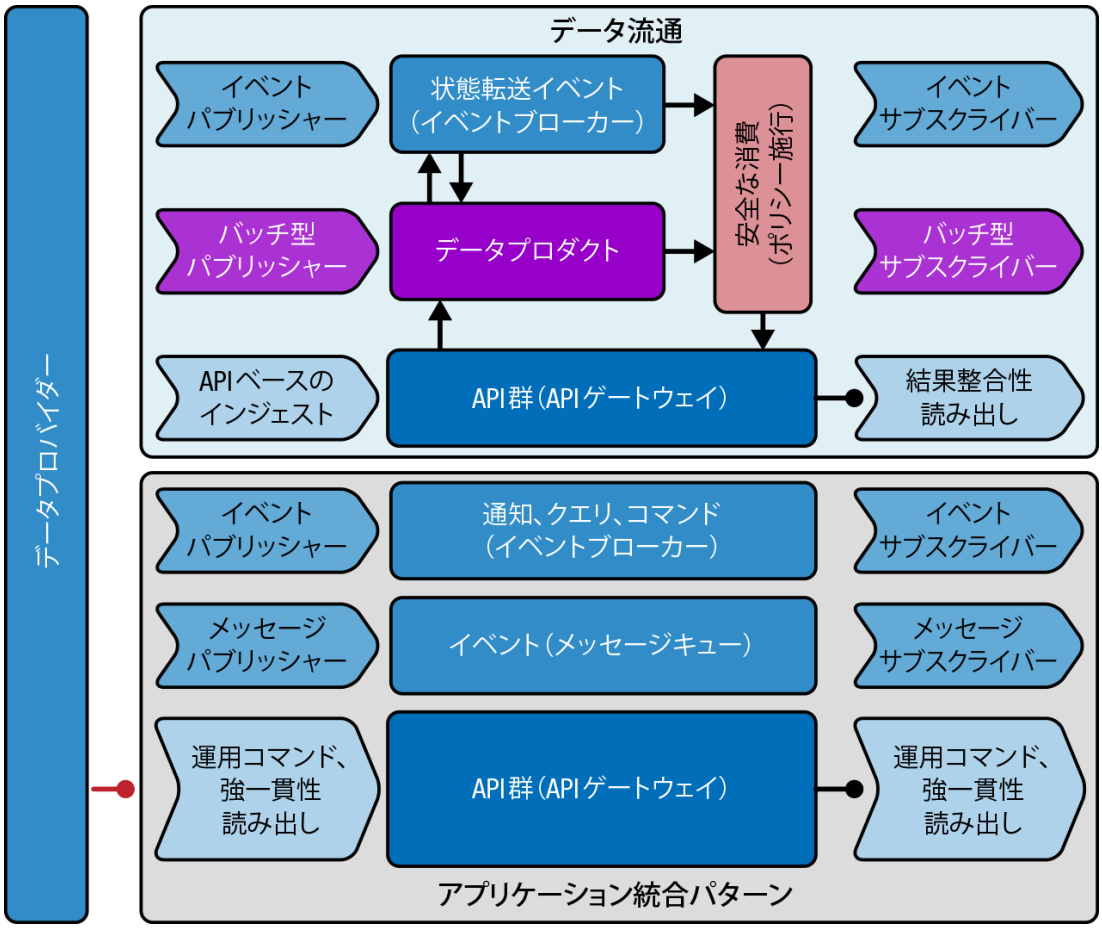

図7-2　各エンタープライズ統合ミドルウェアは、複雑なデータと統合の課題に対処するための一連のデザインパターンを提供する

ご覧のように、データ流通のためのアーキテクチャと、アプリケーション統合のためのアーキテクチャがあります。これは、この2つのアーキテクチャがそれぞれ異なる関心事に焦点を当てているためです。データ流通はデータを利用できるようにして、アプリケーション間でデータを移動させることを目的としています。一方、アプリケーション統合は、アプリケーションコンポーネントなど、アプリケーションをつなげることが目的です。データ流通とアプリケーション統合は、組織の中でどのように管理するかという点が異なります。アプリケーション統合は、エンタープライズソフトウェア開発に重点を置くDevOpsの一環として管理されます。一方で、データ統合は、データの管理とオーケストレーションに重点を置くDataOpsによって管理されます

データ流通とアプリケーション統合をさらに深く掘り下げると、ユースケースやワークロードの特性によって、 検討すべきパターンが異なっていることが分かります。次の項では、図7-2をさらに詳しく見ていき、データ流通とアプリケーション統合のパターンがどのように連動するのかについて、詳細に説明します。その後、データ流通とアプリケーション統合が重なる部分について、どのように管理するのがベストなのかを探ります。

7.1.3　データ流通パターン

一般的に、データ流通パターンは、データを組み合わせて分析する必要がある場合に用いられます。例えば、データ量の多いアプリケーションの開発、データの可視化やアドホックなクエリの実行、機械学習モデルの開発、データマイニングアプリケーションの構築などで使用されます。これらのパターンは、アプリケーションが一連の注文や請求書を処理する必要がある場合など、トランザクション処理のためにデータを一括して移動する場合にも使われます。これらのユースケースに共通するのは、結果整合性読み出しを用いることでスケールアップできるということです。このような場合、アプリケーションデータのコピー（いわゆるデータプロダクト）を操作することになります。

読者のユースケースにおけるデータ流通パターンを検討する際、考慮すべき点が2つあります。1つ目は、運用において変更されてから、その変更を読み出しに反映させるまでの間の遅延時間をどの程度まで許容できるのかを見極めることです。例えば、一日の終わりのデータが必要なアプリケーションと、1分以内でデータを配信する必要があるアプリケーションとでは、要件が異なります。2つ目は、定期的な（例えば月末の）照合や監査が必要かどうかを判断することです。この処理が必要な場合は、バッチ指向のデータ処理パターン（図7-3のパターン2）を使用することをお勧めします。バッチデータ処理は、一定期間内に収集された大量のデータを処理する場合に効率的です。

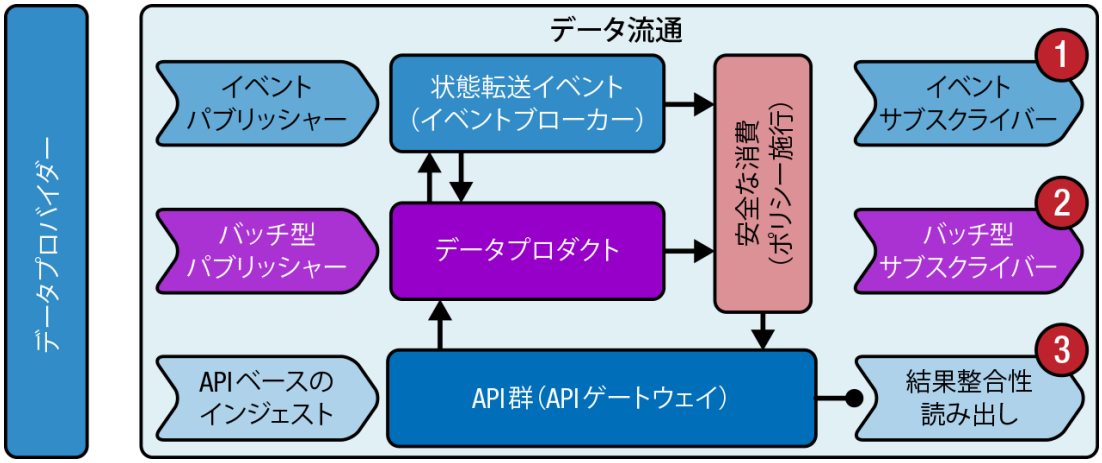

図7-3　データ流通パターン

例えば、運用システムや分散データに加えられた変更を数分から数秒以内にコンシューマに反映させる必要があるなど、スピードが求められる場合は、リアルタイムに近いパターンを検討する必要があります。データを収集するアプリケーションの基本設計にも左右される場合があるため、その選択は難しいものになります。APIベースのパターン（図7-3のパターン1）は、プロバイダーや

コンシューマがインジェストや消費を主導したい場合に適切なソリューションとなります。イベントベースのパターン（図7-3のパターン3）は、状態転送が応答型の場合には良い選択かもしれません。例えば、変更データキャプチャを使用する場合などです。

図7-3に示すように、APIベースやイベントベースのインジェストは、データプロダクトの作成やバッチベースの消費もサポートすることができます。例えば、複数のデータプロダクトに同時にデータを投入するために、ストリーミングプラットフォームを使用して、アプリケーションの状態を転送する場合などです。このパターンは、複数のネットワークやクラウドにまたがるような、分散データプロダクトアーキテクチャを構築する場合に便利です。また、緩やかに変化する次元（SCD）への更新を流通させるときに、データプロダクトとイベントを組み合わせて使うこともできます。SCDを処理するとき、例えば、変更情報を使用して、データにどのような変更が行われたかを検出して、その変更に対応するイベントを生成します。データプロダクトとAPIを組み合わせて、クエリの通信をデータプロダクトに、コマンドの通信を運用システムにルーティングするパターンもあります。このようなCQRSパターンは、運用システムにおけるAPI呼び出しの負荷を軽減します。ここで重要なことは、異なるデータ流通パターンを組み合わせて互いに補い合うことで、データ消費のための包括的なツールキットを構築できるということです。

7.1.4　アプリケーション統合パターン

では、アプリケーション統合アーキテクチャに話を移しましょう。アプリケーション統合アーキテクチャは、スピードと強一貫性が必要不可欠な場合に使用します。前述したように、アプリケーション統合パターンでは、トランザクションの実行、製品の注文、業務プロセスにおける顧客の確認など、ある特定のユースケースやビジネス上の問題のために、それぞれが特定用途に設計された個々のアプリケーションを密に連携させることに重点を置きます。このような場合、記録のシステムから直接情報を取得したいでしょう。これはAPI（図7-4のパターン6）が得意とするところです。APIは、強一貫性のある読み出しやコマンドの直接呼び出し（つまり同期通信）に適した方法です。このパターンは2つのアプリケーション間で直接通信を実現します。APIゲートウェイは、軽いレイヤーとして、これらのアプリケーションの間で仲介を行います。

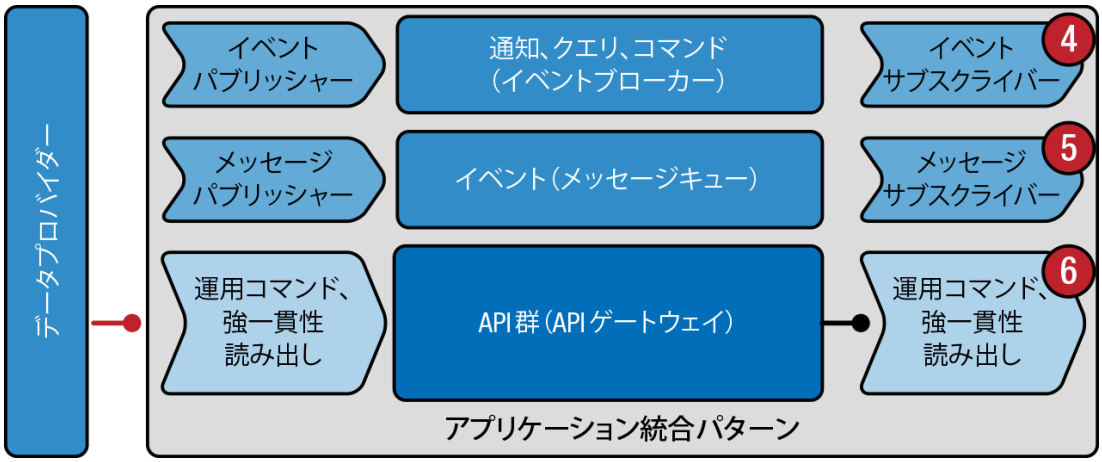

図7-4　アプリケーション統合パターン

非同期のユースケースについては、イベントブローカーとメッセージキュー（図7-4の4と5）という別のパターンを使います。なぜ同期通信よりも非同期通信が良いのでしょうか。それは、アーキテクチャがより弾力的になり、コスト効率が良くなるからです。非同期通信はすべてのシナリオでうまく動作するわけではありませんが、時間的な制約がない場合には、同期通信よりも以下のような利点があります。

- 遅延の長さやスループットの低さといったネットワーク上の問題は、非同期通信には影響しないか、あってもごくわずかです。
- 非同期通信は弾力性があるため、負荷が増加した場合に簡単にスケールアップできます。
- 非同期通信の場合、アプリケーション通信は一時的なネットワーク障害に対して堅牢になります。
- 非同期通信では、再送機構、フォールバック機能、連鎖的な障害を抑止するサーキットブレーカーなど、耐障害性を高めるための重要な機能を実装できます[†1]。

[†1] サーキットブレーカー（https://oreil.ly/XFYvP）は、障害を検知するためのデザインパターンです。サーキットブレーカーは、障害が連続して発生しないようにするロジックをカプセル化したものです。

非同期通信にはいくつかの欠点もあります。通信がリアルタイムで行われていないため、何か問題が発生した場合に、デバッグや調査が難しくなります。リクエスト/レスポンス形式であれば、呼び出しに失敗した場合、通常はすぐにそ

のことに気づけます。しかし、非同期通信では問題が発生してもすぐに気づけない場合があります。また、非同期システムが長期間故障していると、それ自体が新たな問題を発生させるかもしれません。このような問題に対処するために、非同期通信では、アプリケーションコンポーネントを追加しなければならない場合もあります。例えば、非同期メッセージフローを保護し、デッドレターキューやサーキットブレーカーを使って、メッセージが消失するリスクを避けたい場合もあるでしょう。

アプリケーション統合のために、APIとイベントを組み合わせることもできます。例えば、APIに同時に大量のリクエストが殺到するような状況を考えてみてください。過負荷により、リクエストに適切に応答できないため、サービスが停止する場合があります。この問題を解決するには、APIとイベントを組み合わせて、キューを作成し、APIゲートウェイへのリクエストを保持するバッファとして機能させます。このパターンは**キューベースの負荷平準化**（https://oreil.ly/y2IjE）とも呼ばれます。

アプリケーション統合ソリューションを設計するのは難しい作業です。というのも、「正しい」ソリューションは複数存在する場合があるからです。多くの場合、性能、保守性、柔軟性、コスト、弾力性など、さまざまなトレードオフが発生します。このようなことを考慮するには、解決しようとしているビジネス上の問題を深く理解する必要があります。

データプロダクト、API、イベントはさまざまなシナリオやユースケースを実現するために使用されますが、同じ境界内でこれらすべてのパターンを使用する場合、ドメインコンテキストは重なり合います。**図7-1**では、同じアプリケーションがデータ流通とAPI接続の両方に使われています。このことに気づいたところで、話題を意味的一貫性とディスカバラビリティに転換しましょう。

7.1.5　一貫性とディスカバラビリティ

すべての統合サービスとデータサービスを整合させる場合、同時にデータの保護とアプリケーション統合の一貫性も考慮する必要があります。このため、すべてのドメインでアプリケーションデータとインタフェースモデルをドキュメント化する必要があります。これらは十分に記述されていなければならないだけでなく、何よりも重要なのは、すべてのデータ属性がビジネスデータ辞書内の要素に関連付けされている**必要がある**ということです。

異なるドメイン間では、データや統合サービスに関して、意味的な混同が生じるおそれがあるため、コンテキストと意味を把握することは重要です。用語は異なる意味を持つ場合があるため、それぞれのデータ要素を適切なドメインと関連付け、その意味、出所、物理的な保存場所を正しく認識することが不可欠です。この項では、そのためのベストプラクティスをいくつか紹介します。しかし、その前に、さまざまなモデルがどのように関連しているかを振り返ってみましょう。

1章では、概念設計、論理設計、物理設計という、3つの設計フェーズを紹介しました。概念設計はビジネスのエンティティと要素を記述します。論理アプリケーションデータモデルは、エンティティや、エンティティ名、関係、プロパティを含んだ、データストアなどのアプリケーション全体を表したものです。物理アプリケーションデータモデルは、実際に実装された設計を記述します。理想的な状況では、各レイヤーは前のレイヤーを継承し、連結したモデルを形成します。そのため、ビジネスコンセプトがどのように実世界の実装に反映されたのかを、全員が深く理解できます。

実際には、多くのエンジニアが論理モデリングのステップを省略しています。今日、ほとんどのモデルはNoSQLスキーマ設計か非正規化データ構造を使用しているため、論理モデルはかえって邪魔だと考えるエンジニアもいます。このように人気が低下しているにもかかわらず、論理モデルは「最新の」データモデリング手法の一部として再び注目を集めています。論理モデリングの詳細については、Joe Reis氏とMatt Housley氏共著の『データエンジニアリングの基礎』（オライリー・ジャパン、原書“Fundamentals of Data Engineering” O'Reilly）をご一読ください。

図7-5に示すように、3モデルアプローチをデータプロバイダーとデータコンシューマに適用すると、 概念モデルは最初の（一番上の）モデルになります。概念モデルはビジネスコンテキストを整理するために用いられます。ビジネス用

語集はこのようなモデルの出発点になります。論理モデルは概念モデルから導出されます。論理モデルは論理設計を表します。例えば、データベース設計やインタフェースモデルなどです。

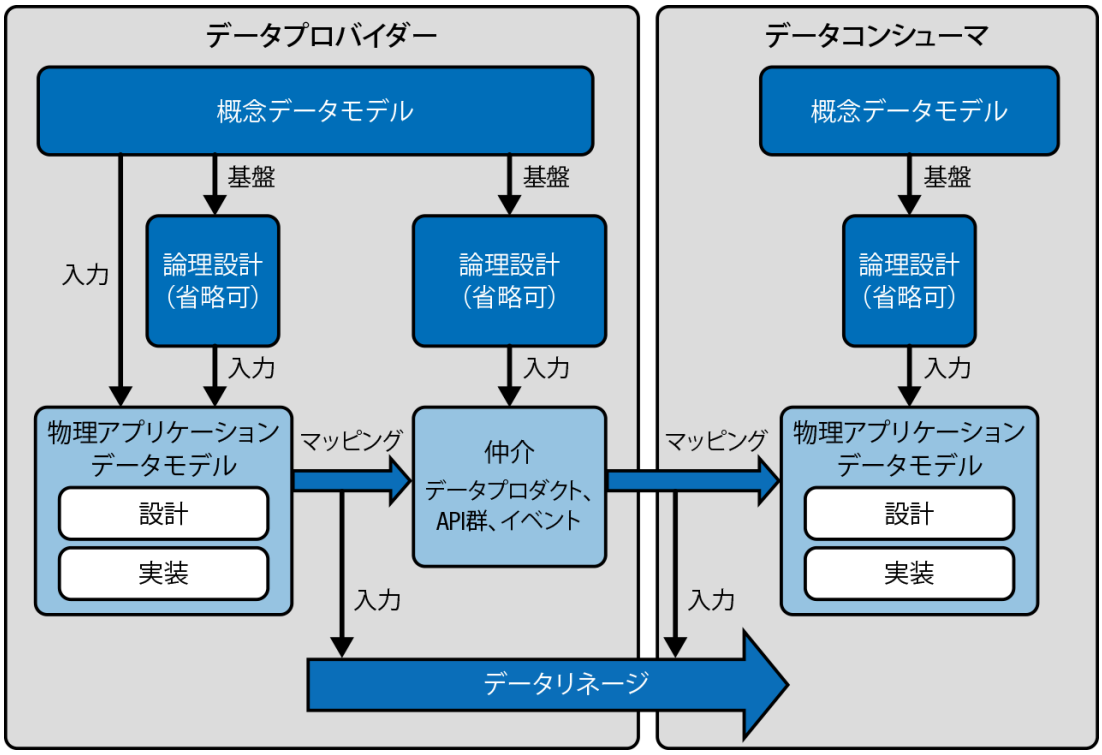

図7-5　概念データモデル、論理データモデル、物理データモデルの属性がすべてつながっているのが理想的。データプロダクトのデータはプロバイダー側で生成されるため、その属性はプロバイダーの概念データモデルにもリンクされる

プロバイダー側では、アプリケーションもインタフェースもドメインの概念データモデルに基づいていることがわかります。データの流通が行われても、コンテキスト（ビジネス用語など）は変わりません。ほかのドメインにデータを提供する場合に異なっているのは、公開されるものがビジネスコンテキスト全体のサブセットになっている点だけです。図7-5に示すように、共有されるコンテキストのサブセットは、データプロバイダーの基盤となるすべてのレイヤーで同じでなければなりません。

論理モデルが存在する場合、データモデリングツールや設計ツールに格納されます。できれば、それぞれの実装に対して、概念データモデルのエンティティやビジネス用語へリンク付けが行われていると良いでしょう。実際に実装された設計を表す物理的なアプリケーションモデルとインタフェースモデルは、これらの設計が作成された環境に格納されているはずです。設計がない場合は、データソースをスキャンし、名前、ファイルサイズ、列などのような技術的な

メタデータを取得することをお勧めします。これらの情報を、例えばデータカタログに保存します。

データモデルを結びつけるとき、同じ意味データが複数の表現を持つことがあるということを忘れないようにしてください。例えば、同じ意味データを異なるファイル形式で保存しているデータプロダクトなどが考えられます。このような場合、ドメインは、すべての技術的属性を同じビジネス要素にマッピングしなければなりません。

消費側にも、概念モデル、論理モデル、物理モデルがあります。提供側との違いは、インタフェースモデルが存在しないことです。データは直接変換され、対象となるアプリケーション設計に統合されるからです。もう1つの違いは、コンテキストが変更されていることです。そのため、提供側のモデルと比較すると、さまざまな種類のモデルが更新され、よりリッチなオブジェクトを持つことになります。

理想的な状態では、すべてのモデルがつながっています。筆者からのお勧めは、概念データモデルからすべてのビジネスコンセプトと関係を取得し、データカタログやナレッジグラフに一元的に格納し、それらをデータプロダクト、API、イベントの論理エンティティの要素（別のリポジトリに格納可能）にリンクし、最後に物理インタフェースモデルの（スキーマ）メタデータに対してすべて照合して検証することです。モデルを簡単につなげるソリューションはありません。理想的な状態に達するには、ツールを選択するか開発し、メタデータを組み合わせて統合する必要があります。カタログや、リポジトリ、ドキュメントポータルから始めるとよいでしょう。さらに先に進むには、一貫性を目標に、すべてのアーキテクチャパターンに対して、コンテキスト間の共通点と相違点について理解してください。そのためには、データ、API、イベントのためにデータモデリングリポジトリとカタログを緊密に連携させる必要があります。

メタデータの統合があまりに難しいと感じる場合は、データカタログから始めて、カタログの要素を物理的なレベルで属性に直接リンクしていく、という実践的な方法があります。この方法を使えば、どの実装に対してどのようなビジネスコンテキストが入力されたかをある程度把握することができます。作業を進めるにつれて、これから管理を行うデータの意味モデルに対して、すでに管理下にあるメタデータと関連付けながら設計することを検討してください。9章では、この話題についてさらに詳しい情報と例を挙げます。

データ検索と意味論の把握は複雑なテーマです。このような問題を解決するために、次のことをお勧めします。

- データモデリング手法を正しく選択することは、メタデータの収集とエンドツーエンドのインサイトにとって非常に重要です。そのため、データプロダクトや統合チームと共同で、データプロダクトモデリングとメタデータの標準を策定することを検討してください。この作業の一環として、適切なツールを選択します。また、社内コミュニティを構築して、データモデリングの実践方法に関する経験やヒントを共有できるようにしてください。
- モデル間の一貫性を確保するために、メタデータに対してデータ品質管理を行うことを検討してください。例えば、コントロールとデータガバナンスの適用レベルにもよりますが、すべての物理属性がビジネス用語にマッピングされているかどうかを確認するのも良いでしょう。マッピングされていない場合は、プロバイダーによるデータやインタフェースの公開を中止します。さらに、カバレッジレベルをレポートすることで、どのドメインがうまく機能しているのか、あるいは機能していないのかを示すこともできます。
- データ管理メタモデルを設計するまでに、あまり期間をおかないようにしてください。メタモデルは、エンティティをどのようにグループ化し、互いに関連付けるべきかを理解する上で助けとなります。また、正しいツールを選択するのにも役立ちます。例えば、論理設計を全面的に採用して、メタデータ主導でアプリケーション設計を自動配備する企業もあります。そのような組織では、より先進的なツールや、ほかのメタデータサービスとの高度な統合が必要になるかもしれません。
- セキュリティとプライバシーへの配慮を忘れないでください。法規制上の理由から、機密データを分類し、タグ付けする必要があるかもしれません。これらのテーマについては、8章で詳しく説明します。
- データモデリングと検索機能について説明し、透明性を保つことが重要です。データ検索がどのように機能するのか、また、ほかのドメインでデータを迅速に見つけ利用するときにどのように役立つかについて、情報を提供することも大切です。デモやトレーニングは必要不可欠です。ガイドラインや導入手順書も成功のためには欠かせません。
- データ管理情報モデルの要素に対して、明確にタクソノミーを定義してください。**エンティティ**や**属性**など、よく使われる用語の定義が曖昧だと、一貫性が保てず、チームメンバー間のコミュニケーションがうまくいかなくなります。

多くの場合、問題はデータそのものの可用性ではなく、データの品質や構造、コンテキスト情報の欠落などに起因します。ドメインチームが真のビジネス価値を提供する際に悪影響を及ぼす可能性があります。このような状況を回避するために、データモデリング、データ検索、意味論的一貫性に対して、戦略とベストプラクティスを事前に明確にしてください。誰がどのようなメタデータを維持する責任があるのかを明らかにし、はっきりとしたガバナンスを確立します。データ品質関連のメタデータを検証する管理プロセスを構築してください。そして、すべてのデータモデリングのメタデータが最新であることを確認するプロセスを策定します。

前述した推奨事項でわかるように、標準化、エンタープライズガイダンス、ドメインチームへの徹底したコミュニケーションは、アーキテクチャにおいて重要な要素です。本章の後半では、このような話を説明したいと思います。この後半では、エンタープライズアーキテクトとして働いていたときの、筆者のベストプラクティスを紹介します。

7.2　変革のための刺激、動機付け、指導

変革を推進し、正しいパターンを選択することは簡単ではありません。そこで、ドメインチームが問題の解決策を開発するのをサポートするために、以下のベストプラクティスをお勧めします。

- 設計セッションを設け、質問して設計を調整できるようにすることで、ドメインアーキテクトをサポートしてください。このようなセッションは、エンタープライズアーキテクト、ソリューションアーキテクト、テクニカルアーキテクト、ビジネス担当者など、ほかの参加者と緊密に連携して開催してください。データと統合は非常に難しいテーマであり、誰もが現状と将来の状況を正しく判断できるわけではありません。また、ワークショップを開催し、アーキテクトを指導、トレーニング、コーチングしてスキルアップさせます。
- プレイブックを開発して、アーキテクトや開発者が選択可能なパターンをまとめてください。それぞれのパターンについて、特徴、達成度、お勧めのテクノロジーサービス、注意点、メリットとデメリットを挙げてください。また、ペルソナごとに異なるプレイブックを用意するのもよいでしょう。例えば、全体的なランドスケープを管理しなければならないエンタープライズアーキテクトと、アプリケーション開発者では、異なるニーズを持っている場合があります。

- 公式に利用が承認されたテクノロジーサービスや製品のリストを公開します。これらのサービスや製品を、プレイブックの各パターンに関連付けます。
- エンタープライズアーキテクチャのコントロールプロセスを調整し、アーキテクトが正しいことを確実に実行できるようにする方法を考えましょう。エンタープライズアーキテクトとして、筆者は、ソリューションインテントテンプレートを変更し、データと統合の項目を追加することを提案しました[†2]。この調整により、ドメインアーキテクトはみな、ソリューション提案の一部として、データと統合のパターンを記述する必要が生じました。これらの提案は、関連するスキルと経験を持つ専門家で構成されるアーキテクチャ設計部門に提出され承認されます。承認できない場合は、アーキテクトに対して修正案を提示します。
- 開発設計の決定事項や実装手順を共有するために、専門センター（コンピテンスセンター）を設置します。このような専門センターは、ドキュメント、ユースケース例、手順、コード例を提供するだけでなく、コーチングや実践に取り組む必要があります。例えば、開発者をサポートするためにZalandoのオープンソースのAPIガイドラインドキュメント（https://oreil.ly/bqGm5）を使えば、開発者が共同作業しやすくなります。

[†2] ソリューションインテントは、スケールドアジャイルフレームワーク（https://oreil.ly/DjG5l）の一部です。ソリューションインテントには、現在と将来の要件、設計、およびソリューションのインテント（意図や、より大きな目的）を記述します。

変革のためのアーキテクチャを構築していると、プラットフォームやサービスが（安定的に）消費・利用できる状態になっていない、という状況に直面することがあります。このような場合、短期的な意思決定がどのような長期的な影響を及ぼすかを考慮することが重要です。例えば、大規模で複雑なソリューションよりも、シンプルな新しいソリューションの方が、後々移行しやすいかもしれません。後で悔やまないような決断をするようにしましょう。また、例外が必要であったり、状況が明確でなかったりする場面に遭遇することもあるでしょう。次の項でそのような状況をいくつか取り上げます。

7.2.1　ドメイン境界の設定

どのデータ流通パターンやアプリケーション統合パターンが自社のニーズに最適かを判断するには、2章で説明したように、ドメイン境界を明確に設定することが重要です。しかし、正確な境界を設定するには合意が必要ですので、簡単な作業ではありません。また、特定のビジネスロジックや抽象化をどこに配置するかを決定することも困難です。なぜなら、機能やデータ変換の中には、プロバイダーとコンシューマのどちらのカテゴリーにも明確にあてはまらないものがあるからです。知識、経験、常識に基づいて決定することになります。

まず、明らかな原則から始めましょう。2章では、ビジネスケイパビリティの分離について学びました。コンテキスト境界内では密結合が許されていますが、ドメインの境界を越える場合は、すべてのインタフェースを分離する必要があります。**図7-6**に示したソリューション設計では、これまでに学んだことをすべて具体化しています。この図では、4つのドメインがデータを流通しています。そのうち2つはソースシステム整合です。そして、残りの2つはコンシューマ整合です。標準サービスを提供するために、データランディングゾーンの汎用ブループリントを使用していることに注目してください。このようにすることで、データのレイヤー化とデータプロダクトの構成方法に一貫性が生まれます。この設計では、データをレイヤー化することで、データプロダクト作成のために入力するローカルデータをドメインが柔軟に管理できるようにしています。しかし、データプロダクト自体は、一元管理されたストレージサービスを通じて流通されます。

もう少しズームインして、ほかにどんな気づきがあるか見てみましょう。

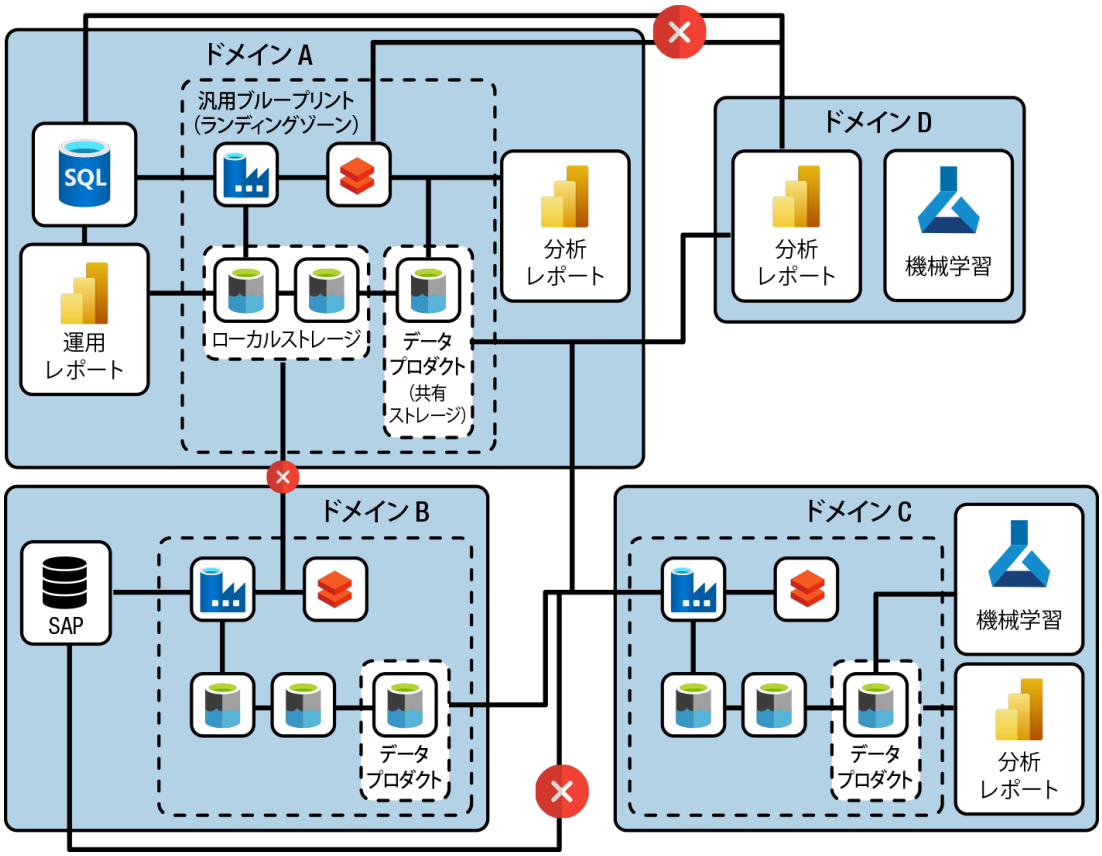

図7-6　ドメイン境界や、どのような統合が許可され、何が禁止されなければならないかを示したソリューション例

ドメインAとドメインBについては、他ドメイン内部の運用アプリケーションを直接利用できません。データプロダクトアーキテクチャから、生の入力データを流通できません。次に、レポートには運用レポートと分析レポートの2種類があることに注目してください。運用レポートは、運用システムを直接使用して、最新かつ最も正確なデータに対してインサイトを得ます。通常、このレポートは低遅延で、データ量も多くありません。システムの主な目的は運用です。一方、分析レポートは、データがコピーされているため、データを組み合わせて多くのクエリを実行できます。

ドメインCはコンシューマです。ドメインCは、ほかの2つのドメインのデータプロダクトを使います。また、汎用ブループリントのデータサービスも使用します。これは、変換ステップを追加するためです。データプロバイダーのデータプロダクトは、ドメインCの要件を完全には満たさないため、データがドメインCのデータプロダクトアーキテクチャに入った後コンシューマ向けに変換されます。

プロバイダーやコンシューマ固有の変換について考えるとき、ビジネスロジックや参照データを相手側の参照データにマッピングする必要がある場合、どの

ように管理するかというのはなかなか難しい問題です。例えば、参照値が細かく、頻繁に変更される場合、データコンシューマがその変更を追いかけるのは困難です。一方で参照データが変更されるたびに、データコンシューマにも影響が及びます。このため、データプロバイダーとデータコンシューマは、粒度について合意する必要があります。データプロバイダーは、きめ細かく詳細なローカルな参照データに対して抽象化や統合を行い、より汎用的で、コンシューマに特化しない、より安定し頻繁に変更されないものにしなければならないでしょう。

境界を設定する際は、ドメインチームにどのようなサービスやプラットフォームを提供しているかにも注意を払ってください。ドメイン内の境界とドメイン間の境界では、ガイダンスも異なります。2つ例を挙げましょう。まず、APIゲートウェイは、アプリケーションコンポーネント間の通信と、ドメインをまたがる通信の両方に使用できます。このゲートウェイは同じ製品やAPI管理プラットフォームであってもかまいません。しかし、ドメインの境界を越える場合、相互運用性は異なります。アプリケーションの境界内では、依存関係が明確にスコープされているため、原則を緩めることもできます。例えば、APIが専門的であったり、バージョンを欠いていたり、適切なレスポンスコードを使用していなかったりする場合でも、それを許容することができます。一方、ドメインの境界を越える場合、ドメイン間の依存関係を管理することが難しくなるため、インタフェースを分離し、消費しやすいように最適化する必要があります。次の例を紹介しましょう。内部アーキテクチャの設計と開発に対しては、柔軟に対応して、より多くの技術的選択肢（サービスや製品を追加することなど）を認めることができます。例えば、ある特定のソフトウェアスイートの一部として提供されているAPIゲートウェイ製品を、アプリケーション内部の通信には使用できます。しかし、ほかのアプリケーションとのインタフェースには使用すべきではありません。

7.2.2　例外処理

ドメインやアプリケーションに、標準化された統合サービスを使うべきではない場合があるのでしょうか。あります。ただし例外は1つだけです。クロスドメインの相互運用性にポイントツーポイントのインタフェースを使用するという例外は、遅延がクリティカルなアプリケーションの場合だけです。このようなアプリケーションは、APIゲートウェイなどの統合コンポーネントを介して分離すると、遅延要件を満たせなくなります。例えば、銀行業界では、支払いをいかに早く処理するかが強く求められています。一般的に、処理時間の合計

は100ミリ秒以下でなければなりません。この限られた時間の中で、多くのことをチェックする必要があります。支払いは一意なのか、同じ支払いを2回処理していないか、口座名義人は支払いを行う資格があるか、この支払いは詐欺ではないのか、受取人がこの支払いを受け取っても問題ないかなどです。アプリケーションがAPIゲートウェイなどを介して分離されている場合、ドメイン間の遅延が増加し、100ミリ秒の時間制限を超えることが考えられます。ポイントツーポイント接続はこの問題を解決します。ただし、このような場合でも、そのほかの標準はすべて守ることをお勧めします。つまり、リネージメタデータが必要な場合は、ポイントツーポイントのソリューションがリネージに関する原則を守るようにしてください。API設計やドキュメントについても同じです。APIゲートウェイが存在しない場合でも、ドメインはデータ管理について責任を負わなければなりません。

包括的な統合とデータアーキテクチャへ移行するのは簡単ではありません。さまざまなチームが緊密に協力し、正しい選択をする必要があります。次の節では、関係するプレイヤーとそのやり取りについて説明します。

7.3　組織の変革

デジタルトランスフォーメーションは、テクノロジー、プロセス、カルチャー、組織の連携を変革する必要があるため、非常に難しいものです。デジタルトランスフォーメーションは、人と実践を中核とする革新的なプロセスです。テクノロジーは変化を実現するための触媒として機能します。データ管理機能の導入も同様です。それは、ビジョンから始まります。ビジョンとは、変革を実現するための完全で総合的なアプローチを含んだ包括的なイメージのことです。このビジョンと全体的な概要設計の策定には、最高データ責任者が先頭に立つべきです。そして、設計が完了したら、正式なプロジェクトやプログラムとして実行に移します。

データ管理機能が実際に導入されるプロジェクトでは、通常以下の3種類のプレイヤーが関与します。

ドメインチーム
ドメインチームは通常、ビジネスオーナー（データオーナー、ドメインオーナー、プロダクトオーナー）、アプリケーションオーナー、ソフトウェア開発者、データエンジニアで構成されます。これらのチームは、互いの要件について明確にコミュニケーションする必要があります。組織の規模や技術的な専門知識のレベルによって、チームの自律性のレベルは変わりま

す。一部の例外を除き、ドメインチームは、ビジネスアプリケーション、データベース、データパイプラインの保守や、正しいデータセットの配信・消費に責任を負います。

中央データプラットフォームチーム

中央データプラットフォームチームは、データに関するコア技術の開発に注力します。通常、開発チームと運用/サイトリライアビリティチームの両方の役割を担っています。中央データプラットフォームチームの主な役割は、ドメインチームにデータ駆動型の機能を提供することです。また、ベストプラクティス、サポート、CI/CD機能、自動テスト機能、プロビジョニングツール、ETLコンポーネントなどのパイプラインツールも提供します。

中央インフラストラクチャー（クラウド）チーム

中央インフラストラクチャーチームは、物理的なハードウェアを管理し、さまざまな組織をサポートするテクノロジーサービスをアセスメントする責任があります。クラウドへ移行すると、このチームの役割は変わります。クラウドに移行した場合、このチームの役割は、サービスを評価し、それらをほかのチームが利用できるようにすることになります。さらにこのチームは、企業ポリシーを策定し、すべてのソリューションが必ず設計に準拠していることを確認する責任も担っています。

図7-7は、これら3つのプレイヤー間の関係を示したものです。一番下に位置するのは、中央インフラストラクチャーチームです。簡単に説明すると、このチームの役割は、組織のすべてのプレイヤーに、汎用的なインフラストラクチャー関連サービスを広く提供することです。データプラットフォームチームがデータプラットフォームの構築を始める際には、インフラストラクチャーチームと緊密に連携してデータ関連サービスを利用できるようにします。インフラストラクチャーチームはコスト、監査、管理、保守性などに焦点を当てますが†3、データプラットフォームチームは、サービスを、使いやすいセルフサービス型のデータ管理機能に変えることに注力します。データプラットフォームチームは、ブローカーの役割を果たして、これらのサービスをそのまま再提供することもあります。しかし、それよりも、より良いエクスペリエンスを提供することを目的として、サービスを統合、変更、結合することのほうが多いです

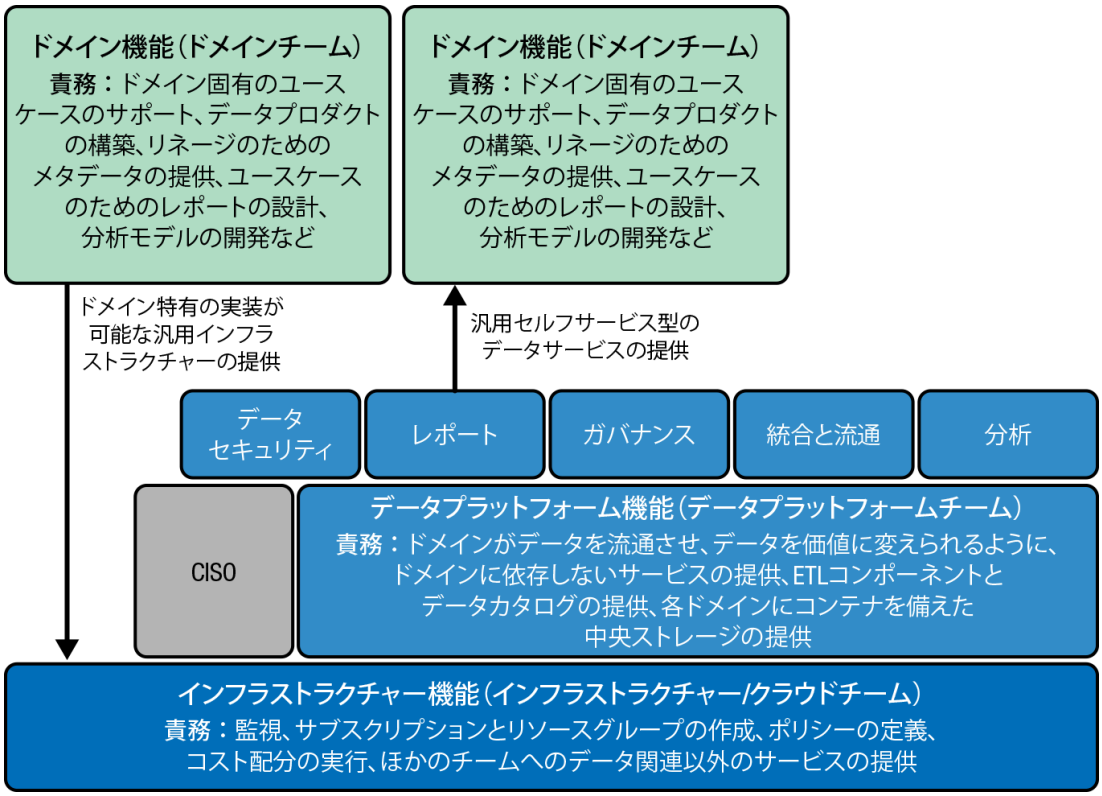

図7-7　ドメインチーム、プラットフォームチーム、インフラストラクチャーチームの連携方法を示した運用モデル

[†3] インフラストラクチャーチームは、すべてのITサービスを監督・管理するために、自らサービスを利用することもあります。このチームが利用するサービスは、IT4ITサービスと呼ばれることもあります。

図7-7の一番上にあるのがドメインチームです。ドメインチームの役割は、ドメインの専門知識を提供し、ドメインの専門知識に基づいて、ビジネスのプロセスと目標をサポートするサービスを構築することです。ドメインチームは、中央インフラストラクチャーチーム（仮想マシンのプロビジョニングが必要な場合など、一般的なITのリクエストのため）と中央データチーム（ビジネスソリューションのために機械学習などのデータサービスが必要な場合）の両方とやり取りすることがあります。

エンタープライズアーキテクトは、チームを超えた交流を推進します。彼らは「エレベーターの乗り方」を知っています[†4]。エンタープライズアーキテクトは、ペントハウス（ビジネス戦略が定義される場所）と機関室（ソフトウェアが構築される場所）をつなぎ、障害となり得るものを発見して取り除きます。ドメインアーキテクトの役割が、関連するドメインの経験を活かしてドメインチームをリードすることであるのに対し、エンタープライズアーキテクトは一般的に、もっと広い範囲を担当します。エンタープライズアーキテクトは、ビジネスとテクノロジーを結びつけます。

[†4] Gregor Hohpe著“The Software Architect Elevator”（O'Reilly）は、すべてのアーキテクトにとって必読の書です。

インフラストラクチャーチームとデータプラットフォームチームを別の部署や論理的な組織にする必要があるのかという質問をよく受けます。筆者はその必要はないと思っています。インフラストラクチャーとデータは異なる関心事であり、別々に管理してもまったく問題ありません。しかし、2つのチームが緊密に連携することで、提供スピードが劇的に向上することがよくあります。インフラストラクチャーチームとデータプラットフォームチームのメンバーで構成されるバーチャルチームを作ってみるのも良いでしょう。

図7-7にも示されているように、中央のデータプラットフォームチームの能力はいくつかのカテゴリーに分けられます。セキュリティ、レポート、ガバナンス、統合と流通、分析などです。これらの領域をすべてカバーする必要があります。さらにスケールアップする場合は、これらの分野に境界を設けて管理することをお勧めします。それぞれの分野は、それ自体がビジネスケイパビリティ（ドメイン）となり、それぞれ固有のプロダクトオーナー、リソース、アプ

リケーション、プロセスを持つことになります。本書で説明するパターンや原則を使用して、これらを分離してもよいでしょう。この図に関する最後の注意点として、セキュリティや運用など、一部の機能はすべてのレイヤーを横断していることを認識しておいてください。これらについて考えるのは全員の役割だからです。

これでメインのプレイヤーがわかりました。しかし、次世代データアーキテクチャを構築するために、実際にはどのように連携するのでしょうか。複雑なデータプラットフォームの提供モデルを詳しく見ていくために、チームトポロジーを検討しましょう。

7.3.1　チームトポロジー

チームトポロジー（https://teamtopologies.com）は、「ビジネスチームやテクノロジーチームを組織化し、素早く業務を遂行するための先進的な取り組みであり、組織設計とチームインタラクションのための実践的で段階的なモデル」を提供するものです。Matthew Skelton氏とManuel Pais氏は同名の書籍の中で、複雑なソリューションを構築する方法について、4つの異なるチームトポロジーを使った知見を紹介しています[†5]。

[†5] Thoughtworks社も、データメッシュを提供する上でチームトポロジーをお勧めしています。

- **ストリーム基準チーム**は、通常、ビジネスドメインの、あるセグメントからの業務の流れとの整合性を重視します。これらのチームは、ほかの種類のチームと頻繁にやり取りし、そのサポートを受けます。
- **イネーブリングチーム**は、ストリーム基準チームが問題を解決するのを支援します。コンサルティングサービスを提供し、機能が不足している部分を突き止めます。
- **複雑サブシステムチーム**は、複雑なタスクを処理することでほかのチームを支援します。通常、高度な数学／計算／技術的専門知識が必要とされる状況で活躍します。
- **プラットフォームチーム**は、そのほかの種類のチームをグループ化したものです。ストリーム基準チームの提供速度を速めるために内部製品を用意します。

これらのチームトポロジーは、前節で説明したモデルにどのようにマッピングできるでしょうか。図7-8に、どのように連携が行われるのかを示します。

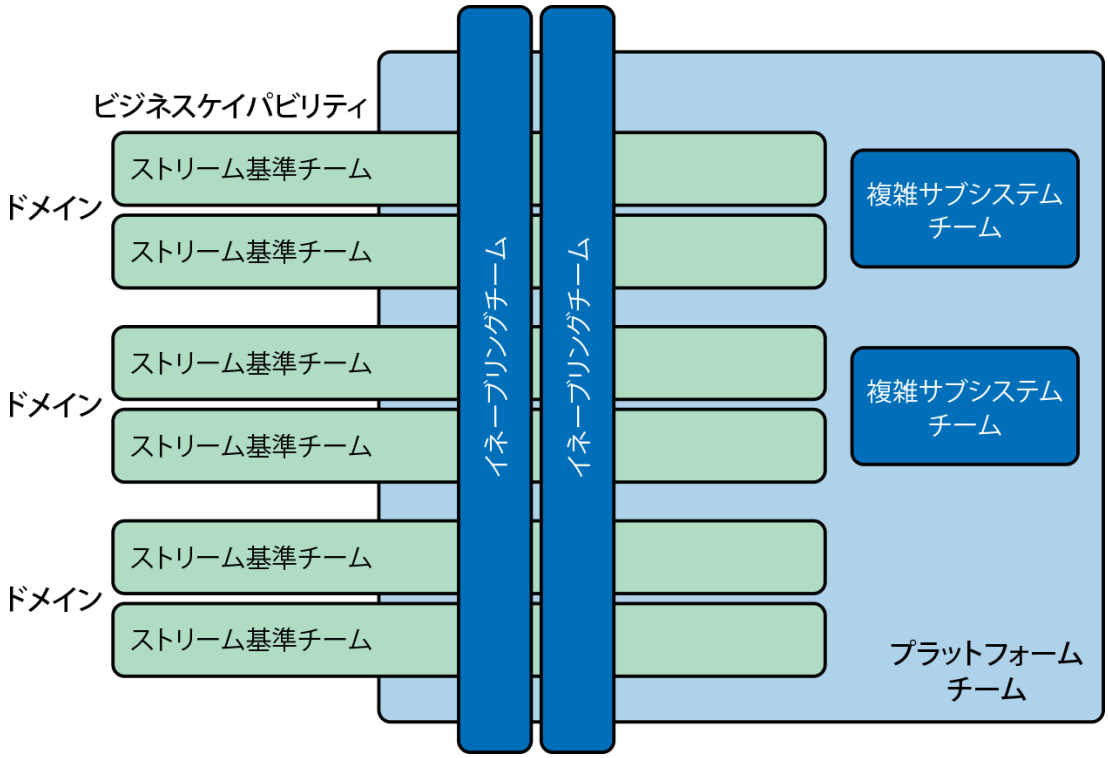

図7-8　ドメインチーム（ストリーム基準チーム）、イネーブリングチーム、複雑サブシステムチーム、プラットフォームチームの連携方法を示すチームトポロジーの考え方

ドメインチームから始めましょう。チームトポロジーでは、ドメインチームはストリーム基準チームです。これまで学んできたように、ドメインチームは、データをデータプロダクトに変え、データプロダクトを価値に変えることで、ビジネス価値を創出します。そのため、データを統合したりパイプラインを開発したりするために、例えばデータ管理サービスを利用します。これらのサービスを提供するのは、データプラットフォームチームとインフラストラクチャーチームです。この2つのチームは、チームトポロジーでは、プラットフォームチームにほぼ相当します。簡単に言うと、このチームの目標は、ほかのチームがデータを使って多くのことを達成できるように、強力なサービスを提供することです。理想的には、ストリーム基準チームは互いに独立して並行して作業しそれぞれが特定のビジネスケイパビリティに集中できるようにします。

そのほかの2つのプレイヤーのチームトポロジーへのマッピングは、あまりはっきりしたものではありません。中央データプラットフォームチームは、ストリーム基準チームとしても、プラットフォームチームとしても考えることができます。データ管理やインフラストラクチャーの観点からは、データプラット

フォームチームはストリーム基準チームのように振る舞います。データプラットフォームチームは（クラウド）インフラストラクチャーチームのサービスを利用し、そのサービスを利用してデータ管理機能を開発するからです。しかし、ドメイン（ストリーム基準）チームの視点で見れば、データプラットフォームチームはむしろプラットフォームチームです。まず、データプラットフォームチームは汎用的なインフラストラクチャーサービスを利用します。これに秘伝のソースを加えて、データ管理機能に変換し、ドメインチームに提供するからです。

ほかのトポロジーはどうでしょうか。図7-8では、ストリーム基準（ドメイン）チームとプラットフォームチームをつなぐイネーブリングチームと、その脇に位置する複雑サブシステムチームが描かれています。これは、データ管理のコンテキストでは、深いレベルの専門知識を必要とする領域がいくつかあるからです。以下がその例です。

- 複雑なデータパイプラインの構築、スクリプトの保守、コードの実行、エンジニアリング活動の遂行などのための統合サービス
- データを保護・管理するセキュリティおよびガバナンスサービス
- データの一貫性と品質を高めるマスターデータ管理とデータ品質サービス
- データを価値に変えるための消費サービス（ビジネスインテリジェンスや機械学習サービスなど）

このような複雑なテーマを、ストリーム基準チームやプラットフォームチームだけで処理しようとすると、膨大な労力が必要となります。したがって、効率化のために、これらの問題をイネーブリングチームに任せることをお勧めします。イネーブリングチームとは、その分野に高い専門性を持ち、協調性のある人々で構成されるグループです。イネーブリングチームは、ほかのチームをサポートし、トレーニングや指導を行います。また、ストリーム基準チームとプラットフォームチームをつなぐ役割も果たします。このようなチームを組織に入れておかないと、例えば、データガバナンスに関するすべての会議にプラットフォームチームが出席しなければならないような事態を招きかねません。

データプラットフォームが成長していくと、複雑サブシステムチームを組織に含めなければならなくなることが多いです。複雑なデータ管理プラットフォームのような大規模システムには、複雑なサブシステム（例えば、機械学習を使用するデータ導入システムなど）が含まれる場合が多く、その構築と運用には深い知識と経験を持つ専門家が必要になります。このようなチームを用意して

おくことで、ストリーム基準チームは、自チームでは専門知識や能力が不足しているような業務を任せることができます。

最後に、さまざまな種類のチームの間には、リソースに関して境界がないことに注意してください。あるチームが価値を提供するために、別のチームに依存しなければならない状況に直面する場合があります。例えば、あるドメインチームがデータプロダクトを効率的に構築するために、ほかのチームのデータモデリングサービスが必要になるかもしれません。このような状況では、チームは待機するか、チームメンバーを交替させてください。**インナーソーシング**とも呼ばれる、このようなリソースの入れ替えや借用モデルは、オープンソース開発のモデルに基づいています。インナーソーシングは、提供を早めるだけでなく、組織全体で知識を強化するのに役立ちます。

7.3.2　組織計画

すべての活動を連動させるためには、正しく計画を立案することが重要です。そのためには、一般的に2つの段階を踏む必要があります。

まず、**スプリント**と**スプリントレビュー**があります。スプリントはスクラムフレームワーク（https://oreil.ly/qhnJN）の一部です。スプリントとは数週間の固定的な期間のことです。この間、チームは目標達成に向けて集中します。それぞれのスプリントの終わりには、スプリントレビューセッションがあり、各チームがその結果をレビューし、次のスプリントの計画と目標を修正します。このセッションでは、チームは、ほかのチームにサポートを求め、次のスプリントで確実に目標を達成するための協力関係やパートナーシップを築くことができます。この段階におけるチーム間のやりとりは、通常1対1の形式です。

2つ目は、多対多で行われる大規模なセッションです。一連のスプリントの後には、多くの場合**プログラムインクリメント（PI）プランニング**[†6]と呼ばれる別のイベントがあります。この対面式のイベントでは、すべてのチームが一堂に会し、次の一連のスプリントに対する目標と抱負を共有します。この活動は、終日にわたることもあります。このイベントの中で、チームは自分たちの仕事をほかの人に示し、フィードバックを受けたり与えたりすることもあります。ある意味、個人的にも組織的にもやりがいのある交流の場ですので、100人以上が参加し、ケーキが振る舞われても不思議ではありません。以前、リードアーキテクトとして働いていたときは、このPIプランニングミーティングを、将来のアーキテクチャを提案したり、参加者に影響を与えたり、アーキテ

クチャ設計に関するフィードバックを収集したりする場としても利用していました。

[†6] PIプランニングの詳細については、Scaled Agile Framework（https://oreil.ly/R7Sd7）を参照してください。

新しいアーキテクチャの導入は簡単ではありません。計画、実行、調整、組織改革が必要です。将来のアーキテクチャやダイアグラムは大切ですが、それは変化していくものです。組織を構成する上で、トポロジーやスクラムなどの方法論では、アーキテクトは実装を通してチームを導き、推進するリーダーとして振る舞わなければなりません。実践的で現実的でありながら、ビジョンを示さなければなりません。最も重要なことは、アーキテクトはすべての人を鼓舞し、後に続くように導くことです。

7.4　まとめ

中央集権型から非中央集権型への移行は、今日のデータ駆動型企業において大変革です。連合をサポートするために必要となる、新たな業務のやり方には軋轢が生じることが予想されます。新しいアーキテクチャの実装と提供にはリスクが伴います。このため、基本的な活動から始めることをお勧めします。まずは、ビジネスケイパビリティから始めましょう。ドメイン（ストリーム基準）チームを明確にし、組織と整合させます。非中央集権化には、ドメインの全体像を把握することが必要です。次に、アーキテクチャの構築に着手します。そのためには、中央のインフラストラクチャーチームとデータプラットフォームチームが手を取り合って対応する必要があります。少しずつ進めてください。最初からメタデータ管理に細心の注意を払い、適切なレベルの自動化とセルフサービス機能を実装します。このような初期段階において、エンタープライズアーキテクトは重要な役割を担っています。エンタープライズアーキテクトは、組織のさまざまな部分を見守り、コミュニケーションし、導き、つなげる役割を担っています。最初のフェーズの成果と達成度によって、さらに先に進めるかどうかが決まります。

土台が整った後は、提供するドメインを増やすことで変革プログラムを拡大させることができます。まず、どのようなパターンを提供したいか、ニーズを調査しながら検討してください。次に、選択可能な多くのテクノロジーを使っ

て、どのようにそれを実現するかを決定します。標準化を進めつつ、同時に柔軟性を持つようにしましょう。テクノロジーの乱立やコストの爆発を避けるため、サービスはインフラストラクチャーチームとプラットフォームチームの管理下に置いてください。チームとのつながりを維持し、成功体験を紹介し、フィードバックを受けるために、コミュニティの場を活用しましょう。

この章では、データ流通とアプリケーション統合のパターンを、上空からの視点で解説しました。また、同時に、本書で説明していく組織変革に関連した、さまざまな要素やプレイヤーの概要についても説明しました。次の章でも高空を飛行したままですが、時には複雑なテーマをより深く掘り下げていきます。まずはデータガバナンスとデータセキュリティから説明します。次にメタデータ管理とマスターデータ管理について学びます。その後、ヘリコプターでアーキテクチャの消費側を飛び回ります。シートベルトを締め、離陸に備えてください。